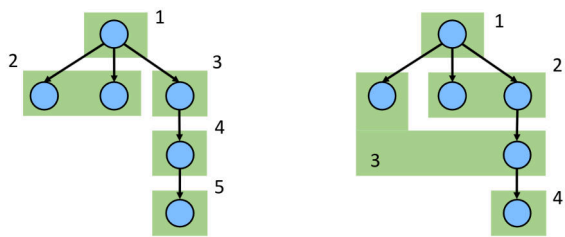


What is T2 for this graph?



T₂ will be 5 with this scheduling (we have 5 time steps)

T₂ will be 4 with this scheduling (we have 4 time steps)

DAG bounds to exec time:

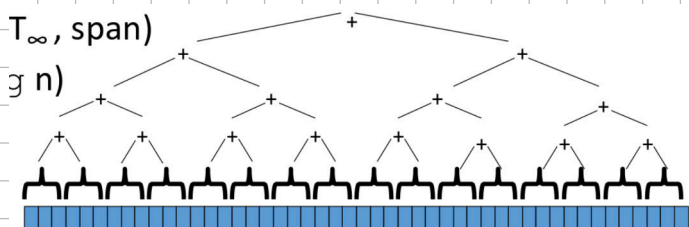
Lower bound: Work law: $T_p \geq T_1/p$, span law: $T_p \geq T_\infty$, $T_p \geq \max(T_1/p, T_\infty) \rightarrow$ no scheduler can do better

Upper bound: Parallelizable work: T_1/p , DAG dependencies: $O(T_\infty)$, $T_p \leq T_1/p + O(T_\infty)$

$\rightarrow$ Java's ForkJoin guarantees an expected time $T_p = O(T_1/p + T_\infty) \rightarrow$

uses work-stealing scheduler $\rightarrow$ ensures good load balancing (T_1/p is not artificially inflated) and reduces scheduling overhead

Parallel Patterns:



Summing an array: Sequential / $T_1 = O(n)$, Parallel / $T_\infty(\text{span}) = O(\log n)$, Parallelism = $O(n/\log n)$

$\rightarrow$ In D&C, there's a way to find the left/rightmost element satisfying a certain property $\rightarrow$ Reductions

$\rightarrow$ produce single answer from collection via associative operator (max, count, sum, product, ..., not median, subtraction, ...)

$\rightarrow$ recursive results don't have to be single numbers or strings, can also be arrays or objects with fields

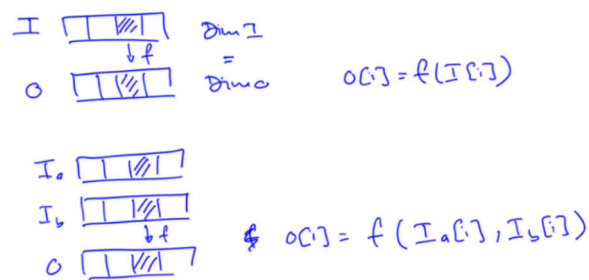
$\rightarrow$ still, there are sequential dependencies, like how $\text{arr}[i]$ can depend on computations of $\text{arr}[i-1]$

$\rightarrow$ Maps: applies function to all elements of collection, producing same size output $\rightarrow$ example: squaring all elements of an array

$\rightarrow$ what if function takes multiple inputs? Zip map (any pythoners?) $\rightarrow$ applies function to multiple collections element-wise, output size still equal to input size

$\rightarrow$ example: element-wise addition of 2 arrays

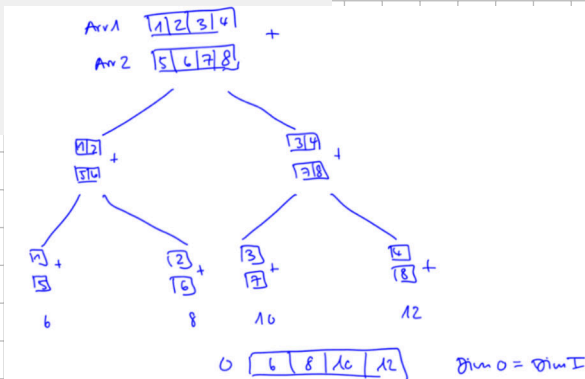
Now, the larger array sum example:



```
protected void compute(){
    if(hi-lo < SEQUENTIAL_CUTOFF) {
        for(int i = lo; i < hi; i++)
            res[i] = arr1[i] + arr2[i]; // 2 inputs, 1 output
    } else {
        int mid = (hi + lo) / 2;
        VecAdd left = new VecAdd(lo, mid, res, arr1, arr2);
        VecAdd right = new VecAdd(mid, hi, res, arr1, arr2);
        left.fork();
        right.compute();
        left.join();
    }
}
```

$\leftarrow$ compute

$\leftarrow$ use map



$\rightarrow$ maps and reductions are common in parallel algorithms

$\rightarrow$ reduction: $O(\log n)$ parallel time, map: $O(\log n)$ with D&C

Linked Lists: Can we parallelize map or reduce over linked lists? It's useful for per-element operations, but reaching the data still takes $O(n)$ compared to even balanced trees ($O(\log n)$)

→ parallel algorithms are still bottlenecked by sequential parts (Andriuk), but there are ways to parallelize seemingly sequential parts

→ Example: Prefix sum problem: given $\text{int}[n]$ input, produce $\text{int}[n]$ output with $\text{output}[i] = \text{input}[0] + \dots + \text{input}[i]$

Sequential version:

```
int[] prefix_sum(int[] input){
    int[] output = new int[input.length];
    output[0] = input[0];

    for(int i = 1; i < input.length; i++){
        output[i] = output[i-1] + input[i];
    }

    return output;
}
```

→ work = $O(n)$, span = $O(n)$ → seq. algorithm, but there is another with span = $O(\log n)$

How would you compute this in parallel?

finer-grained example:

input	6	4	16	10	16	14	2	8
output	6	10	26	36	52	66	68	76

+10

input	6	4	16	10	16	14	2	8
output	6	10	26	36	52	66	68	76

+10

+30

input	6	4	16	10	16	14	2	8
output	6	10	26	36	52	66	68	76

+10

+30

input	6	4	16	10	16	14	2	8
output	6	10	26	36	52	66	68	76

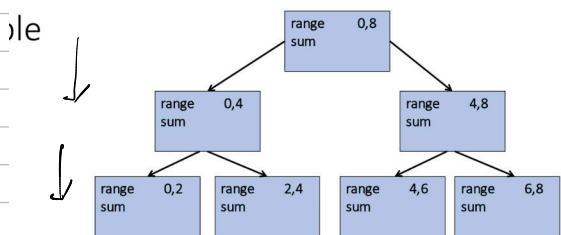
input	6	4	16	10	16	14	2	8
output	6	10	26	36	52	66	68	76

Parallel prefix sum: 2 passes with $O(n)$ work, $O(\log n)$ span → total work, span same

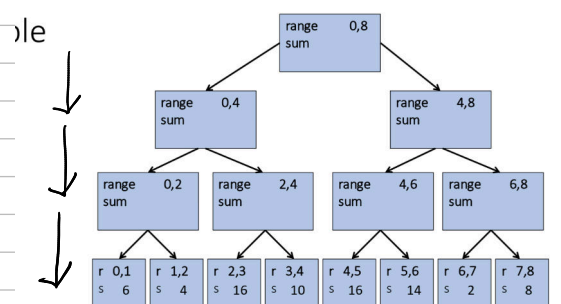
→ parallelism = $n/\log n$

→ First pass builds tree bottom-up, second traverses it top-down

Example:

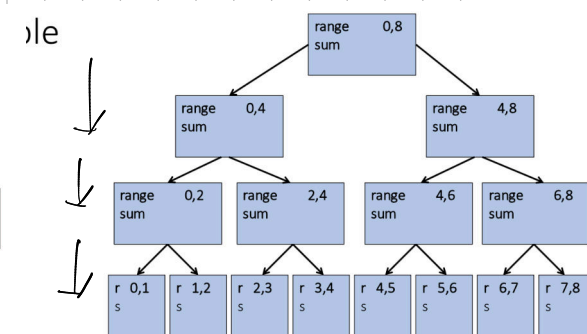


input	6	4	16	10	16	14	2	8
output								

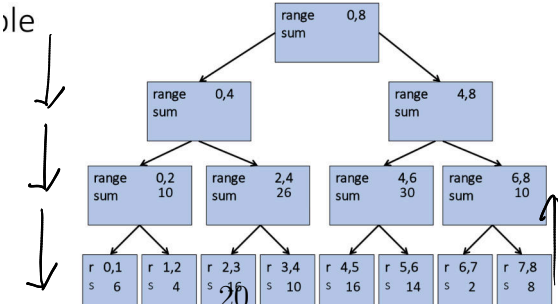


input	6	4	16	10	16	14	2	8
output								

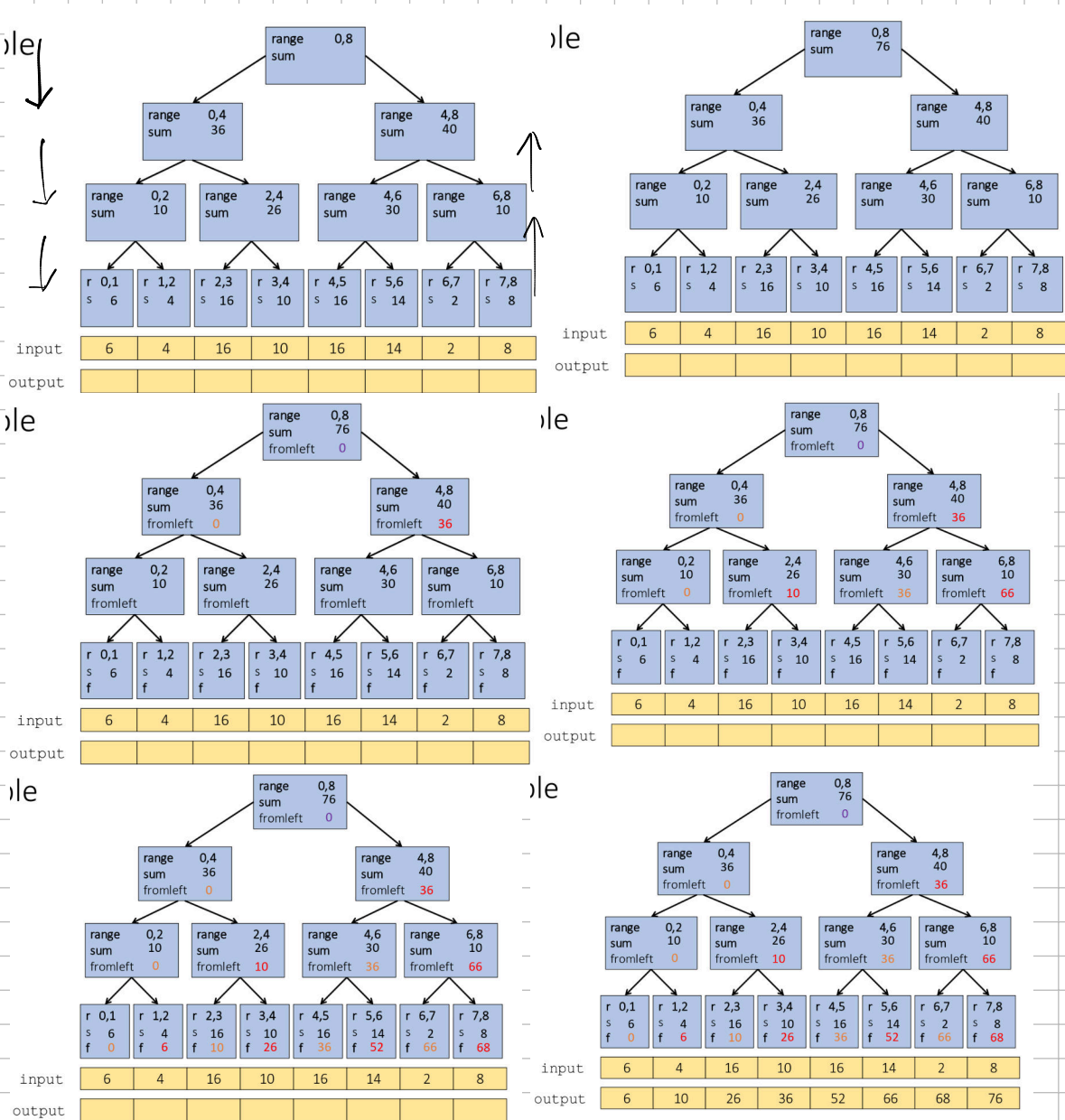
input	6	4	16	10	16	14	2	8
output								



input	6	4	16	10	16	14	2	8
output								



input	6	4	16	10	16	14	2	8
output								



Algorithm:

- Up:** Build bin. tree where: root has sum of range $[x, y]$, if node has sum of $[lo, hi]$ and $hi > lo$, left child has sum of $[lo, middle]$, right child has sum of $[middle, hi]$, a leaf has sum of $[i, i+1]$ (input $[i]$)
 → easy FJ computation with $O(n)$ work, $O(\log n)$ span, combine results by building bin. tree with range sums
- Down:** Pass down value fromleft: root given fromleft of 0, node takes its fromleft value and passes its left child the same fromleft, but passes its right child its fromleft + its left child's sum (from pt. 1)
 → at leaf for arr. position i , output $[i] = \text{fromleft} + \text{input}[i]$
 → easy FJ computation: reverse tree from pt. 1, produce no result, leaves assign to output
 → Invariant: fromleft is sum of elements left of node's range
 → same work, same span as pt. 1

In Java (below):

```
main:
// Step 1: build tree with sums
UpTask root = new UpTask(input, 0, input.length);
fjpool.invoke(root);

// Step 2: traverse same tree with fromLeft
fjpool.invoke(new DownTask(root, 0, input, output));
```

Step 1: Up pass - build recursion tree and compute sums

```
class UpTask extends RecursiveTask<Integer>
{
    ...
    compute(){
        if (leaf) { sum = ...
        } else {
            left = new UpTask(...)
            right = new UpTask(...)
            left.fork();
            right.compute();
            left.join();
            sum = left.sum + right.sum;
        }
        return sum;
    }
}
```

Example: input [17, 4, 6, 8, 11, 5, 13, 19, 0, 24]
f: is elt > 10
output [17, 11, 13, 19, 24]

Using parallel prefix:

1. Parallel map to compute bit vector for true elements

input [17, 4, 6, 8, 11, 5, 13, 19, 0, 24]
bits [1, 0, 0, 0, 1, 0, 1, 1, 0, 1]

2. Parallel prefix sum on bit vector

bitsum [1, 1, 1, 1, 2, 2, 3, 4, 4, 5]

3. Parallel map (with condition) to produce the output

output [17, 11, 13, 19, 24]

	Best / expected case work
1. Pick a pivot element	$O(1)$
2. Partition all the data into:	$O(n)$
A. The elements less than the pivot	
B. The pivot	
C. The elements greater than the pivot	
3. Recursively sort A and C	$2T(n/2)$

Easy: Do the two recursive calls in parallel

- Work: unchanged of course $O(n \log n)$
- Span: now $T(n) = O(n) + 1T(n/2) = O(n)$
- So parallelism (i.e., work / span) is $O(\log n)$

Partition all the data into:

- A. The elements less than the pivot
- B. The pivot
- C. The elements greater than the pivot

Example: Step 1: pick pivot as median of 3 elements

Step 2: Down pass - propagate prefix sums (fromLeft) down the tree

```
class DownTask extends RecursiveAction {
    ...
    DownTask(UpTask node, int fromLeft, int[] input, int[] output) {
        this.node = node;
        this.fromLeft = fromLeft;
        this.input = input;
        this.output = output;
    }
    compute() {
        if (leaf) { sequentialPrefix(...)
        } else {
            DownTask left = new DownTask(node.left, fromLeft, input, output);
            DownTask right = new DownTask(node.right, fromLeft + node.left.sum, input, output);
            left.fork();
            right.compute();
            left.join();
        }
    }
}
```

node: range [lo,hi), sum, left child, right child

Rule:
left child gets: fromLeft of the parent
right child gets: fromLeft of the parent + sum of the left subtree

Generalized: this is a pattern that reappears

examples: min/max of all elements to the left of i, is there elt. of i with some property?, count of elements left of i satisfying some property

Pack: given an array input, produce array output containing only elements s.t. f(elt) is true (some condition)

Parallelizable? Finding elements for output is easy, getting them in the right place feels hard

Pack comments:

Steps 1, 2 can be combined into one pass by using a different base case for the prefix sum, has no effect on asymptotic complexity

can also combine step 3 into down pass of prefix sum $\rightarrow$ no effect on asymptotic complexity

$O(n)$ work, $O(\log n)$ span

helps to parallelize QuickSort (expected time $O(n \log n)$)

Can we improve on $O(\log n)$ speedup with infinite processors?

Not in place, with extra storage we can use packs to parallelize the partition

Pack is $O(n)$ work, $O(\log n)$ span

Pack elements < pivot into left side of aux array

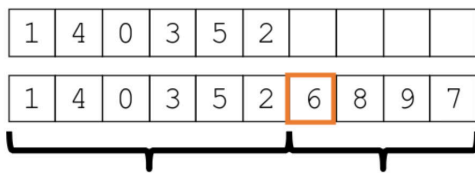
Pack elements > pivot into right side of aux array

with cleverness, we could do both packs at once $\rightarrow$ no effect on asymptotic complexity

with $O(\log n)$ span for partition, total span for QuickSort is $T(n) = O(\log n) + 1T(n/2) = O(\log^2 n) \Rightarrow$ parallelism = $O(n \log n) / O(\log^2 n) = O(n / \log n)$

8 1 4 9 0 3 5 2 7 6

Steps 2a, 2c (combinable): pack less than, then pack greater than into a second array (fancy parallel prefix, we don't need to know how)



Step 3: Two recursive sorts in parallel → can sort back into original array (like mergesort)

Shared Memory & Locks:

Managing state: Field immutability is the best option, make fields immutable when possible

→ Isolated mutability: only one thread can access mutable data

→ Mutable shared data: Data can change all threads can access it → protect state by allowing one thread access at a time

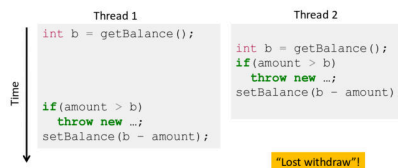
Example:

Correct code in a single-threaded world

```
class BankAccount {
    private int balance = 0;
    int getBalance() { return balance; }
    void setBalance(int x) { balance = x; }
    void withdraw(int amount) {
        int b = getBalance();
        if (amount > b)
            throw new WithdrawTooLargeException();
        setBalance(b - amount);
    }
    // other operations like deposit, etc.
}
```

Race condition

Interleaved `withdraw(100)` calls on the same account
• Assume initial balance == 150



Incorrect fix: "fix" bad interleaving by rearranging/repeating operations

```
void withdraw(int amount) {
    if (amount > getBalance())
        throw new WithdrawTooLargeException();
    // maybe balance changed
    setBalance(getBalance() - amount);
}
```

→ fixes nothing, narrows problem by one statement, now a negative balance is possible

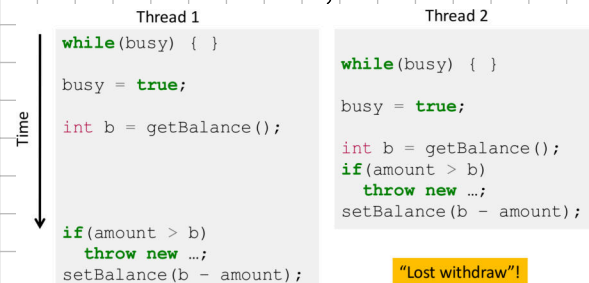
→ Same fix: allow at most 1 thread to withdraw at a time (exclude other simultaneous operations like deposit too)

→ This is called **mutual exclusion** (mutex): one thread using a resource means another thread must wait → **critical sections**

→ Programmer must implement critical sections because compiler can't tell what interleavings are bad

```
class BankAccount {
    private int balance = 0;
    private boolean busy = false;
    void withdraw(int amount) {
        while (busy) { /* "spin-wait" */ }
        busy = true;
        int b = getBalance();
        if (amount > b)
            throw new WithdrawTooLargeException();
        setBalance(b - amount);
        busy = false;
    }
    // deposit would spin on same boolean
}
```

implementing mutex ourselves
won't help



→ we need a lock → primitive with atomic ops. "new" (makes a new, unheld lock), "acquire" (blocks if lock is currently held, once it is not held, it makes the lock held), "release" (makes the lock not held, if ≥ 1 threads are blocked on it, exactly 1 will acquire it) → needs atomic way to check if held and set, for now just take locks as a primitive

→ Required Properties: **Safety** → at most one thread is executing the CS at a time, **Liveness** → minimally: acquire must finish in finite time when no process executing CS

→ the synchronized block uses each object's **internal/intrinsic lock** → there are external locks in the java.util.concurrent.locks library for different scenarios → example: ReentrantLock is literally the lock we know, but explicit

→ Example below

```

class BankAccount {
    private int balance = 0;
    private Lock lk = new ReentrantLock(); // One lock for each account, initially not held
    ...
    void withdraw(int amount) {
        lk.lock(); // May block ← entering CS
        int b = getBalance();
        if (amount > b)
            throw new WithdrawTooLargeException();
        setBalance(b - amount);
        lk.unlock(); ← leaving CS
    }
    // deposit would also acquire/release lk
}

```

What if an exception occurs?

Without try-finally, if an exception occurs before unlock(), the lock is never released

```

class BankAccount {
    private int balance = 0;
    private Lock lk = new ReentrantLock(); // One lock for each account, initially not held
    ...
    void withdraw(int amount) {
        lk.lock(); // May block ← entering CS
        try {
            int b = getBalance();
            if (amount > b)
                throw new WithdrawTooLargeException();
            setBalance(b - amount);
        } finally {
            lk.unlock(); ← leaving CS
        }
    }
}

```

If an exception (like WithdrawTooLargeException) occurs, the lock will still be released

→ Other issues: **Incorrectness** ← using different locks for withdraw, deposit, since balance is the shared field being protected and **mutex** requires the same lock for different methods, **Poor performance** ← Same lock for all accounts, since there are no simultaneous operations on different accounts

```

if (lk.tryLock(100, TimeUnit.MILLISECONDS)) { // Wait up to 100ms
    try {
        // Critical section
    } finally {
        lk.unlock();
    }
} else {
    // Lock not acquired, handle accordingly ← Avoiding long waits or deadlocks, implementing fallback strategies
}

```

→ lock() acquires the lock and blocks indefinitely until available

→ tryLock() attempts to acquire without blocking, returns true if successful, false if lock held, supports timeouts

Code example using tryLock()

→ continuing our code example:

```

public class MyThread implements Runnable {
    private Lock lock;

    public MyThread(Lock lock) {
        this.lock = lock;
    }

    public void run() {
        if (lock.tryLock(2000, TimeUnit.MILLISECONDS)) {
            try {
                printMessage("Lock acquired");
            } finally {
                printMessage("Lock released");
                lock.unlock();
            }
        } else {
            printMessage("Lock not acquired - return");
            return;
        }
    }
}

```

```

public void setBalance(int x) { .. }

public int getBalance() { .. }

public void withdraw(int amount) {
    ..
    b = getBalance();
    ..
    setBalance(b - amount);
    ..
}

public void deposit(int amount) {
    ..
    b = getBalance();
    ..
    setBalance(b + amount);
    ..
}

```

→ we need **reentrancy**:

→ consider getBalance, setBalance (they are public)

→ If they don't acquire same lock as deposit and withdraw, a race between setBalance and withdraw can cause incorrectness

→ if they do acquire the same lock, withdraw would block forever to acquire a lock it already has

Implementation with separate setBalances:

```

int setBalance1(int x) {
    balance = x;
}

int setBalance2(int x) {
    lk.lock();
    setBalance1(x);
    lk.unlock();
}

void withdraw(int amount) {
    lk.lock();
    ...
    setBalance1(b - amount);
    lk.unlock();
}

int setBalance(int x) {
    lk.lock();
    balance = x;
    lk.unlock();
}

void withdraw(int amount) {
    lk.lock();
    ...
    setBalance(b - amount);
    lk.unlock();
}

```

Reentrant lock: recursive, remembers currently holding thread and a count

→ If the lock is acquired, the count is set to 0 → if the current holder's code calls acquire, it doesn't block, instead incrementing the count

→ on release, the count is decremented, if it is 0, the lock becomes not held

→ Given a reentrant lock, our previous example works

Synchronized vs. Lock API:

Aspect

synchronized

java.util.concurrent.locks

Lock handling

Automatic (JVM managed)

Manual (lock()/unlock())

Lock release

Automatic

Must be done explicitly (finally block)

Scope

Method/block scoped

Flexible (across multiple methods)

Waiting behavior

Thread blocks indefinitely

tryLock() avoids blocking

Interruptibility

Not interruptible while waiting

lockInterruptibly() supports interrupt

Flexibility

Low

High

Complexity

Simple, less error-prone

More complex, error-prone

Deadlock risk

Lower

Higher (if unlock forgotten)

Typical use

Simple synchronization

Advanced concurrency control

Race Conditions: Happens when program correctness depends on interleaving → problem is usually some thread seeing an intermediate state causing incorrectness → Data Races and bad Interleavings are defined on page 5

Example: Bounded Stack

```
public class Stack <E> {
    E[] array;
    int index;

    public Stack(int entries) {
        // hack to generate a generic array,
        // initialized with NIL values
        array = (E[]) new Object[entries];
        index = 0;
    }

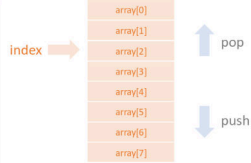
    public class Stack <E> {
        ...

        E peek() {
            E ans = pop();
            push(ans);
            return ans;
        }
    }
}
```

```
public class Stack <E> {
    ...
    synchronized boolean isEmpty() {
        return index == 0;
    }

    synchronized void push(E val) throws StackFullException {
        if (index == array.length)
            throw new StackFullException();
        array[index++] = val;
    }

    synchronized E pop() throws StackEmptyException {
        if (index == 0) throw new StackEmptyException();
        return array[--index];
    }
}
```

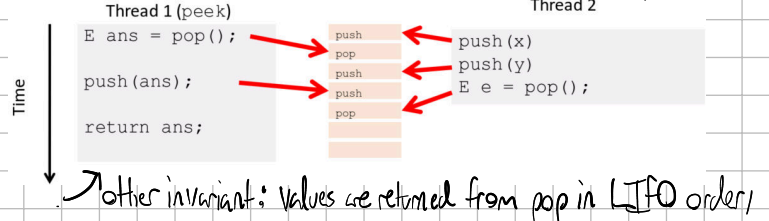
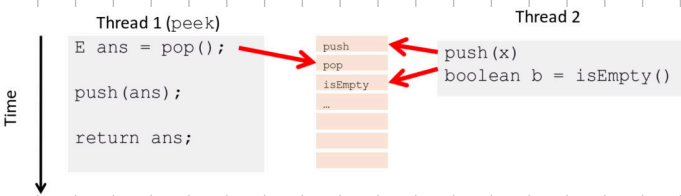


→ peek does not modify stored data → it's a reader, not a writer, and as pop/push are synchronized, there are no data races

→ inconsistent intermediate state → bad interleavings, stack invariants violated

→ each operation is thread-safe, but the system is not behaviorally safe

→ **Invariant:** if there has been a push and no pop, isEmpty returns false → current peek() violates this and following invariant



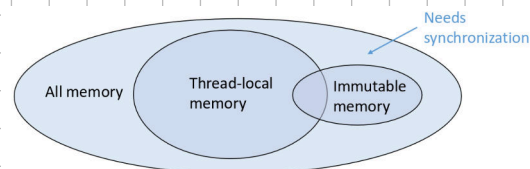
→ other invariant: values are returned from pop in LIFO order

```
class Stack<E> {
    ...
    synchronized E peek() {
        E ans = pop();
        push(ans);
        return ans;
    }
}
```

→ fix: synchronize peek(), secure it from bad interleavings → larger CS, reentrant lock for push/pop calls

General guidelines:

→ each mem. location in your program must be at least one of thread-local, immutable, or synchronized



→ anywhere where you don't need inter-thread communication: keep it thread-local, this ends up being most fields in parallel programs

→ minimize mutation for shared memory, simultaneous reads require no synchronization

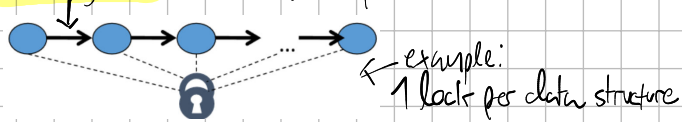
→ now, the guidelines for shared, mutable memory:

→ **no data races:** never allow 2 threads to read/write or write/write the same location at the same time (#0)
→ necessary, not sufficient for correctness

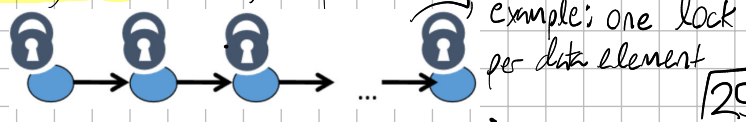
→ any location needing synchronization needs an (always held) lock when reading/writing the location → lock guarding the location (#1)
→ same lock can/should guard multiple locations, often, the guard is intrinsic, you can put it on this in object methods
→ consistent locking isn't enough, bad interleavings are still possible, it also isn't required for correctness (lock-free later)

→ start with coarse-grained and move to fine-grained only if contention on coarser locks breaks correctness (#2)

→ coarse-grained: more objects per lock



→ fine-grained: fewer objects per lock



→ coarse-grained is easier to implement in general, especially so for operations that access multiple locations or operations that modify data structure shape

→ fine-grained allows for more simultaneous access → better performance, coarse-grained would block more

→ do not introduce race conditions or do expensive computations / IO in CS (#3)

→ if CS too long: performance loss from blocking other threads, if CS too short: bugs by exposing intermediate state, performance loss from thread switch overhead (more frequent) and cache thrashing

Example:

```
synchronized(lock) {  
    v1 = table.lookup(k);  
    v2 = expensive(v1);  
    table.remove(k);  
    table.insert(k, v2);  
}
```

→ we want to change hashtable value for a key without removing it (assume lock guards whole table)

→ CS too long → lock during expensive call

```
synchronized(lock) {  
    v1 = table.lookup(k);  
}  
v2 = expensive(v1);  
synchronized(lock) {  
    table.remove(k);  
    table.insert(k, v2);  
}
```

← CS too short:
if another thread were
to update, it would be lost

CS just right: →
if there was an
update, try again

```
done = false;  
while(!done) {  
    synchronized(lock) {  
        v1 = table.lookup(k);  
    }  
    v2 = expensive(v1);  
    synchronized(lock) {  
        if(table.lookup(k) == v1) {  
            done = true;  
            table.remove(k);  
            table.insert(k, v2);  
        }  
    }  
}
```

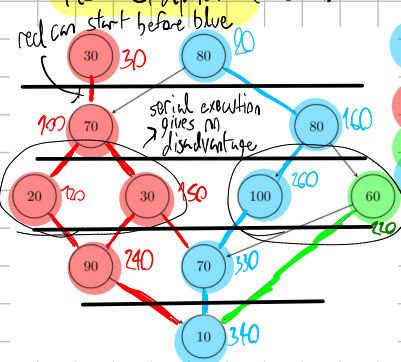
→ think in terms of what operations need to be atomic (#4)

→ Op. is atomic if no other thread can see it partially executed → atomic = "appears indivisible" → abstract data type ops. should be atomic

→ make CS just long enough to preserve atomicity, then design lock to implement CS correctly

More Part 1 exam tasks:

Task Graphs: (FS21)



- (a) Markieren Sie den kritischen Pfad des Task-Graphen in Abbildung 1. Was ist die Ausführungszeit des kritischen Pfads?
- (b) Was ist die minimale Anzahl Prozessoren, die nötig sind um den Task-Graphen in Abbildung 1 so schnell wie möglich auszuführen? Wie hoch ist die Ausführungszeit?

- (2) Mark the critical path of the task graph in Figure 1. What is the execution time of the critical path?
- (4) What is the minimum number of processors necessary to execute the task graph in Figure 1 as quickly as possible? What is the execution time?

→ serial execution leads to disadvantage → we need to add a processor for max. efficiency
⇒ answer to b): 3 processors, execution time = critical path = 340

- (c) Angenommen Sie haben einen Weg gefunden einen bisher sequentiellen Arbeitsschritt zu parallelisieren. D.h. Sie dürfen aus einem Knoten zwei Knoten machen, welche dieselben eingehenden und ausgehenden Kanten haben wie der originale Knoten, aber nur die Hälfte der Zeit benötigen. Welchen Knoten würden Sie aufteilen, um die höchste Beschleunigung zu erzielen? Würde dies die minimale Anzahl Prozessoren von Aufgabe b) für die schnellstmögliche Ausführung dieses neuen Graphen ändern? Wie hoch ist nun die Ausführungszeit?

- (3) Assume you discovered a way to parallelize a sequential task. That means you are allowed to split one node into two nodes having the same incoming and outgoing edges as the original node, but require only half the time. Which node would you split to yield the highest speedup? How would your choice affect the minimal number of processors in task b) required for the fastest execution of the graph? What is the execution time now?

trivial!

not so trivial! we keep the critical path marked, now think of execution on a timeline, because task A must finish before B can start

also less trivial! My intuition would have been to split the 100, leading to a critical path of 80 → 80 → 70 → 10 = 300, BUT we would need 4 processors. Therefore, splitting the first or last 80 node is optimal because we would have the same critical path with 3 processors

→ General tip: color your paths in, draw on the tree

Threads/wait/notify: (I am pulling this and the next section from the exam prep lecture)

→ Notes: usually you won't have to handle exceptions and will lose points if you oversynchronize

Nehmen Sie an Ihnen ist die folgende nicht-thread-sichere Implementation einer Queue mit den folgenden Methoden:

- `public boolean isEmpty()`, die true zurück gibt falls die Queue leer ist und sonst false
- `public void enqueue(Object o)`, die das Objekt o zur Queue hinzufügt
- `public Object dequeue()`, die ein Element aus der Queue entfernt und dieses zurück gibt oder null zurück gibt fall die Queue leer ist

Assume you are given a non-thread safe implementation of a Queue class with the following methods:

- `public boolean isEmpty()` which returns true if the queue is empty, and false otherwise
- `public void enqueue(Object o)` which adds an object o to the queue
- `public Object dequeue()` which returns an object removed from the queue or null if the queue is empty

→ all of these methods are given use them!

Your goal is to implement a thread safe version of a parallel queue, but **do not** introduce unnecessary synchronization. Fill in the following skeleton with code, such that the `ParallelBlockingQueue` class behaves as specified in the comments. [For this task, you do not need to handle exceptions]

Solution:

```
1 public class ParallelBlockingQueue {
2     private Queue q = new Queue();
3
4     public synchronized boolean isEmpty() {
5         return this.q.isEmpty();
6     }
7
8     public synchronized void enqueue(Object o) {
9         // ...
10    }
11
12    public synchronized Object dequeue() {
13        // ...
14    }
15
16    // ...
17
18    // ...
19 }
```

isEmpty?

```
// Testet ob die Queue leer ist.
// @return true falls die Queue leer ist und sonst false.
/* Checks if the queue is empty.
 * @return true if the queue is empty, and false otherwise. */
```

simple enough!

Ihr Ziel ist es eine thread-sichere Version einer parallelen Queue `queue` zu implementieren, aber **keine** unnötige Synchronisation einzuführen. Vervollständigen Sie das folgende Codeskelett, so dass sich `ParallelBlockingQueue` verhält wie in den Kommentaren spezifiziert. [In dieser Aufgabe müssen Sie Exceptions nicht behandeln.]

Threads & Wait/Notify

Solution:

```
1 public class ParallelBlockingQueue {
2     private Queue q = new Queue();
3
4     public synchronized boolean isEmpty() {
5         return this.q.isEmpty();
6     }
7
8     public synchronized void enqueue(Object o) {
9         this.q.enqueue(o);
10        this.notifyAll();
11    }
12
13    public synchronized Object dequeue() {
14        // ...
15    }
16
17    // ...
18
19 }
```

enqueue?

```
// Fügt ein Element zur Queue hinzu.
// @param o - das Objekt das hinzugefügt werden soll.
/* Adds an object to the queue.
 * @param o - the object to be added to the queue. */
```

→ release intrinsic lock

longer, less trivial one: (just enjoy the walk-through)

Wir modellieren eine vereinfachte Postfiliale als Thread. Die Postfiliale bedient Kunden. Jeder Kunde hat eine eindeutige Ticketnummer und wird als separater Thread modelliert.

Vervollständigen Sie das folgende Code-Skelett, indem Sie der Beschreibung in den Kommentaren folgen. Sie dürfen neue Felder hinzufügen, aber **nicht** neue Methoden. Ihre Lösung sollte `notify()`, `notifyAll()`, `wait()` und das `synchronized`-Schlüsselwort verwenden um Ihr Programm korrekt zu synchronisieren, aber Sie sollen **keine** unnötige Synchronisation einführen. In dieser Aufgabe müssen Sie Exceptions nicht behandeln.

Consider a simplified post office, modeled as a thread, which serves its customers. Each customer has a unique ticket number and is modeled as a separate thread.

Complete the following code skeleton according to the description in the comments. You are **allowed** to add new fields, but you **cannot** add new methods. Make sure that your solution is properly synchronized by using `notify()`, `notifyAll()`, `wait()` and the `synchronized` keyword where required, but **do not** introduce unnecessary synchronization. For this task, you do not need to handle exceptions.

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
```

```
1. erhöht die angezeigte Nummer (displayedNumber) und
2. informiert alle Kunden
3. wartet auf den Kunden mit der angezeigten Nummer
4. bedient den Kunden
5. bedient den Kunden

/* The Post Office repeats forever:
 * 1. increases the displayed number and informs all the Customers
 * 2. waits for the Customer with the displayed number to come
 * 3. serve the Customer */

public void run() {
    while (true) {
        // ...
        displayedNumber++;
        notifyAll();
        // ...
    }
}
```

dequeue?

```
// Entfernt ein Objekt von der Queue und gibt es zurück. Wenn die
// Queue leer ist, wartet die Methode bis ein Element vorhanden
// ist. Wenn mehrere Threads gleichzeitig warten, ist die Reihenfolge
// in welcher diese returnen beliebig.
// @return das Objekt, dass von der Queue entfernt wurde.
/* Removes and returns an object from the queue. If the queue is empty,
 * then the method waits until some object can be dequeued. If multiple
 * threads wait at the same time, the order in which they return is arbitrary.
 * @return the object removed from the queue. */
```

→ gotta have the while loop to retry

```
// Bedient den Kunden c.
// Diese Methode sollte in PostOffice Thread ausgeführt werden.
/* Serves the Customer c.
 * This method should be executed in the PostOffice thread. */
private synchronized / none void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
public synchronized / none int getDisplayedNumber() {
    return displayedNumber;
}
```

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;
```

```
...

Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}
```

```
// Der Kunde wartet bis seine/ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The customer waits until his/her ticket number is displayed
 * by the post office, then he/she goes to the post office
 * to be served. We assume that the post office can only serve
 * a single customer at a time. */
public void run() {
    // ...
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
 * 1. increases the displayed number and informs all the Customers
 * 2. waits for the Customer with the displayed number to come
 * 3. serve the Customer */
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
```

```
public void run() {
    while (true) {
        synchronized(this) {
            displayedNumber++;
            notifyAll();
            // ...
        }
    }
}
```

```
private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
 * 1. increases the displayed number and informs all the Customers
 * 2. waits for the Customer with the displayed number to come
 * 3. serve the Customer */
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
```

```
public void run() {
    while (true) {
        synchronized(this) {
            displayedNumber++;
            notifyAll();
            // ...
        }
    }
}
```

```
private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
```

```
public void run() {
    while (true) {
        // ...
        displayedNumber++;
        notifyAll();
        // ...
    }
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
 * 1. increases the displayed number and informs all the Customers
 * 2. waits for the Customer with the displayed number to come
 * 3. serve the Customer */
```

```
private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
 * 1. increases the displayed number and informs all the Customers
 * 2. waits for the Customer with the displayed number to come
 * 3. serve the Customer */
```

```
private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
```

```
public void run() {
    while (true) {
        synchronized(this) {
            displayedNumber++;
            notifyAll();
            // ...
        }
    }
}
```


Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;

    public Customer customerAtTheCounter;

    public void run() {
        while (true) {
            synchronized(this) {
                displayedNumber++;
                notifyAll();
                while(customerAtTheCounter == null) {
                    wait();
                }
            }
        }
    }
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
* 1. increases the displayed number and informs all the Customers
* 2. waits for the Customer with the displayed number to come
* 3. serve the Customer */

private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
* 1. increases the displayed number and informs all the Customers
* 2. waits for the Customer with the displayed number to come
* 3. serve the Customer */

private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    while(ticketNumber != post.getDisplayedNumber) {
        post.wait();
    }
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    while(ticketNumber != post.getDisplayedNumber) {
        post.wait();
    }
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    synchronized(post) {
        while(ticketNumber != post.getDisplayedNumber) {
            post.wait();
        }
    }
}
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;

    public Customer customerAtTheCounter;

    public void run() {
        while (true) {
            synchronized(this) {
                customerAtTheCounter = null;
                displayedNumber++;
                notifyAll();
                while(customerAtTheCounter == null) {
                    wait();
                }
            }
        }
    }
}
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;

    public Customer customerAtTheCounter;

    public void run() {
        while (true) {
            synchronized(this) {
                customerAtTheCounter = null;
                displayedNumber++;
                notifyAll();
                while(customerAtTheCounter == null) {
                    wait();
                }
            }
            serve(customerAtTheCounter);
        }
    }
}
```

Threads & Wait/Notify

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;

    public Customer(PostOffice post, int ticketNumber) {
        this.post = post;
        this.ticketNumber = ticketNumber;
    }

    public void run() {
        while(ticketNumber != post.getDisplayedNumber) {
            post.wait();
        }
    }
}
```

Threads & Wait/Notify

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;

    public Customer(PostOffice post, int ticketNumber) {
        this.post = post;
        this.ticketNumber = ticketNumber;
    }

    public void run() {
        synchronized(post) {
            while(ticketNumber != post.getDisplayedNumber) {
                post.wait();
            }
        }
    }
}
```

Threads & Wait/Notify

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;

    public Customer(PostOffice post, int ticketNumber) {
        this.post = post;
        this.ticketNumber = ticketNumber;
    }

    public void run() {
        synchronized(post) {
            while(ticketNumber != post.getDisplayedNumber) {
                post.wait();
            }
        }
        post.customerAtTheCounter = this;
    }
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
* 1. increases the displayed number and informs all the Customers
* 2. waits for the Customer with the displayed number to come
* 3. serve the Customer */

private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Die Postfiliale wiederholt unendlich lange:
// 1. erhöht die angezeigte Nummer (displayedNumber) und
// informiert alle Kunden
// 2. wartet auf den Kunden mit der angezeigten Nummer
// 3. bedient den Kunden
/* The Post Office repeats forever:
* 1. increases the displayed number and informs all the Customers
* 2. waits for the Customer with the displayed number to come
* 3. serve the Customer */

private void serve(Customer c) {
    System.out.println("Customer " + c + " is being served...");
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    while(ticketNumber != post.getDisplayedNumber) {
        post.wait();
    }
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    synchronized(post) {
        while(ticketNumber != post.getDisplayedNumber) {
            post.wait();
        }
    }
}
```

```
// Der Kunde wartet bis Seine/Ihre Ticketnummer von der
// Postfiliale angezeigt wird und geht dann zur Filiale um
// bedient zu werden. Wir nehmen an, dass die Postfiliale nur
// einen Kunden gleichzeitig bedienen kann.
/* The Customer waits until his/her ticket number is displayed
* by the post office, then he/she goes to the post office
* to be served. We assume that the post office can only serve
* a single customer at a time. */

public Customer(PostOffice post, int ticketNumber) {
    this.post = post;
    this.ticketNumber = ticketNumber;
}

public void run() {
    synchronized(post) {
        while(ticketNumber != post.getDisplayedNumber) {
            post.wait();
        }
    }
    post.customerAtTheCounter = this;
}
```

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
    public Customer customerAtTheCounter;
    public void run() {
        while (true) {
            synchronized(this) {
                customerAtTheCounter = null;
                displayedNumber++;
                notifyAll();
            }
            while(customerAtTheCounter == null) {
                wait();
            }
            serve(customerAtTheCounter);
            // customerAtTheCounter = null;
        }
    }

    private void serve(Customer c) {
        System.out.println("Customer " + c + " is being served...");
    }

    public int getDisplayedNumber() {
        return displayedNumber;
    }
}
```

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;

    public Customer(PostOffice post, int ticketNumber) {
        this.post = post;
        this.ticketNumber = ticketNumber;
    }

    public void run() {
        synchronized(post) {
            while(ticketNumber != post.getDisplayedNumber)
                post.wait();
        }
        post.customerAtTheCounter = this;
    }
}
```

We're stuck!

Threads & Wait/Notify

```
public class PostOffice extends Thread {
    private int displayedNumber;
    public Customer customerAtTheCounter;
    public void run() {
        while (true) {
            synchronized(this) {
                customerAtTheCounter = null;
                displayedNumber++;
                notifyAll();
            }
            while(customerAtTheCounter == null) {
                wait();
            }
            serve(customerAtTheCounter);
            // customerAtTheCounter = null;
        }
    }

    private void serve(Customer c) {
        System.out.println("Customer " + c + " is being served...");
    }

    public int getDisplayedNumber() {
        return displayedNumber;
    }
}
```

```
public class Customer extends Thread {
    private PostOffice post;
    private int ticketNumber;

    public Customer(PostOffice post, int ticketNumber) {
        this.post = post;
        this.ticketNumber = ticketNumber;
    }

    public void run() {
        synchronized(post) {
            while(ticketNumber != post.getDisplayedNumber) {
                post.wait();
            }
            post.customerAtTheCounter = this;
            post.notifyAll();
        }
    }
}
```

→ tips: synchronize on a reference shared by all threads, not on wrapper types like Integer, Character
wait/notify in sync. block, notifyAll() when multiple threads could be waiting, notify() when it's only one (writing to something that could change waiting condition)
wait in a while loop!

Fork/Join (hardest so far IMO)

Fork/Join Framework

6. Sie erhalten einen langen String, und Ihre Aufgabe ist es, zu zählen, wie häufig ein Buchstabe im String vorkommt (gemäß dem Programm in Abbildung 3).

You are given a large string and your task is to count the number of occurrences of a given character as shown in Figure 3.

```
1 public static Integer CharCount(String input, Character target) {
2     int count = 0;
3     for (int i = 0; i < input.length(); i++) {
4         Character c = input.charAt(i);
5         if (c == target) {
6             count++;
7         }
8     }
9     return count;
10 }
```

CharCount("Hello World!", 'o') → 2
CharCount("Hello World!", 'l') → 3
CharCount("Hello World!", 'a') → 0

```
1 public class CharCounterTask extends RecursiveTask<Integer> {
2
3     int start;
4     int length;
5     String input;
6     Character target;
7
8     public CharCounterTask(String input, Character target, int start, int length) {
9         this.start = start;
10        this.length = length;
11        this.input = input;
12        this.target = target;
13    }
14
15    protected Integer compute() {
16        .....
17        .....
18        .....
19        .....
20        .....
21    }
```

```
1 public CharCounterTask(String input, Character target, int start, int length) {
2     this.start = start;
3     this.length = length;
4     this.input = input;
5     this.target = target;
6 }
```

```
1 protected Integer compute() {
```

Basic structure?

```
1 public class CharCounterTask extends RecursiveTask<Integer> {
2
3     int start;
4     int length;
5     String input;
6     Character target;
7
8     public CharCounterTask(String input, Character target, int start, int length) {
9         this.start = start;
10        this.length = length;
11        this.input = input;
12        this.target = target;
13    }
14
15    protected Integer compute() {
16        .....
17        .....
18        .....
19        .....
20        .....
21    }
```

```
1 public CharCounterTask(String input, Character target, int start, int length) {
2     this.start = start;
3     this.length = length;
4     this.input = input;
5     this.target = target;
6 }
```

```
1 protected Integer compute() {
```

Base Case?
~ seq. cutoff, doesn't have to be 100

```
1 public class CharCounterTask extends RecursiveTask<Integer> {
2
3     int start;
4     int length;
5     String input;
6     Character target;
7
8     public CharCounterTask(String input, Character target, int start, int length) {
9         this.start = start;
10        this.length = length;
11        this.input = input;
12        this.target = target;
13    }
14
15    protected Integer compute() {
16        .....
17        .....
18        .....
19        .....
20        .....
21    }
```

```
1 public CharCounterTask(String input, Character target, int start, int length) {
2     this.start = start;
3     this.length = length;
4     this.input = input;
5     this.target = target;
6 }
```

```
1 protected Integer compute() {
2     if (this.length <= 100) {
3         int count = 0;
4         for (int i = this.start; i < this.start + this.length; i++) {
5             if (this.input.charAt(i) == this.target) {
6                 count++;
7             }
8         }
9         return count;
10    } else {
```

Base case?

```
1 public CharCounterTask(String input, Character target, int start, int length) {
2     this.start = start;
3     this.length = length;
4     this.input = input;
5     this.target = target;
6 }
7
8 protected Integer compute() {
9     if (this.length <= 100) {
10        int count = 0;
11        for (int i = this.start; i < this.start + this.length; i++) {
12            if (this.input.charAt(i) == this.target) {
13                count++;
14            }
15        }
16        return count;
17    } else {
18        int halfSize = (this.length) / 2;
19        CharCounterTask firstHalf =
20            new CharCounterTask(
21                this.input, this.target, this.start, halfSize);
22        CharCounterTask secondHalf =
23            new CharCounterTask(
24                this.input, this.target,
25                this.start + halfSize, this.length - halfSize);
26        return firstHalf.compute() + secondHalf.compute();
27    }
28 }
```

How do we split?

```
1 public CharCounterTask(String input, Character target, int start, int length) {
2     this.start = start;
3     this.length = length;
4     this.input = input;
5     this.target = target;
6 }
7
8 protected Integer compute() {
9     if (this.length <= 100) {
10        int count = 0;
11        for (int i = this.start; i < this.start + this.length; i++) {
12            if (this.input.charAt(i) == this.target) {
13                count++;
14            }
15        }
16        return count;
17    } else {
18        int halfSize = (this.length) / 2;
19        CharCounterTask firstHalf =
20            new CharCounterTask(this.input, this.target, this.start, halfSize);
21        CharCounterTask secondHalf =
22            new CharCounterTask(this.input, this.target,
23                this.start + halfSize, this.length - halfSize);
24        return firstHalf.compute() + secondHalf.compute();
25    }
26 }
```

How do we split?


```

public CharCounterTask(String input, Character target, int start, int length) {
    this.start = start;
    this.length = length;
    this.input = input;
    this.target = target;
}

```

How do we split?

```

protected Integer compute() {
    if (this.length <= 100) {
        int count = 0;
        for (int i = this.start; i < this.start + this.length; i++) {
            if (this.input.charAt(i) == this.target) {
                count++;
            }
        }
        return count;
    } else {
        int halfSize = (this.length) / 2;

        CharCounterTask firstHalf =
            new CharCounter(this.input, this.target, this.start, halfSize);

        CharCounterTask secondHalf =
            new CharCounter(this.input, this.target,
                this.start + halfSize, this.length - halfSize);

        firstHalf.compute();
        secondHalf.compute();
        return firstHalf.getCount() + secondHalf.getCount();
    }
}

```

Now do the forking!

2x fork and 2x join is also okay

```

public CharCounterTask(String input, Character target, int start, int length) {
    this.start = start;
    this.length = length;
    this.input = input;
    this.target = target;
}

protected Integer compute() {
    if (this.length <= 100) {
        int count = 0;
        for (int i = this.start; i < this.start + this.length; i++) {
            if (this.input.charAt(i) == this.target) {
                count++;
            }
        }
        return count;
    } else {
        int halfSize = (this.length) / 2;

        CharCounterTask firstHalf =
            new CharCounter(this.input, this.target, this.start, halfSize);

        CharCounterTask secondHalf =
            new CharCounter(this.input, this.target,
                this.start + halfSize, this.length - halfSize);

        firstHalf.fork();
        secondHalf.fork();
        firstHalf.join();
        secondHalf.join();
        return firstHalf.getCount() + secondHalf.getCount();
    }
}

```

```

public CharCounterTask(String input, Character target, int start, int length) {
    this.start = start;
    this.length = length;
    this.input = input;
    this.target = target;
}

protected Integer compute() {
    if (this.length <= 100) {
        int count = 0;
        for (int i = this.start; i < this.start + this.length; i++) {
            if (this.input.charAt(i) == this.target) {
                count++;
            }
        }
        return count;
    } else {
        int halfSize = (this.length) / 2;

        CharCounterTask firstHalf =
            new CharCounter(this.input, this.target, this.start, halfSize);

        CharCounterTask secondHalf =
            new CharCounter(this.input, this.target,
                this.start + halfSize, halfSize);

        firstHalf.fork();
        secondHalf.fork();
        firstHalf.join();
        secondHalf.join();
        return firstHalf.getCount() + secondHalf.getCount();
    }
}

```

Watch out when splitting the array!

this.length = 8
halfSize = 4



this.length = 7
halfSize = 3



→ I think now you see why this is 12 points!

→ Tips: Memorize structure, remember order of fork() and join()
Split correctly, pay attention if length or indices are used
Understand RecursiveTask, RecursiveAction
2x.fork and 2x.join or 1.fork/join and 1.compute both correct

The Java Memory Model: (JMM)

Motivation (below):

Motivation

```

class C {
    private int x = 0;
    private int y = 0;

    Thread 1
        x = 1; (A)
        y = 1; (B)
    }

    Thread 2
        int a = y; (C)
        int b = x; (D)
        assert(b >= a);
    }
}

```

Can this fail?

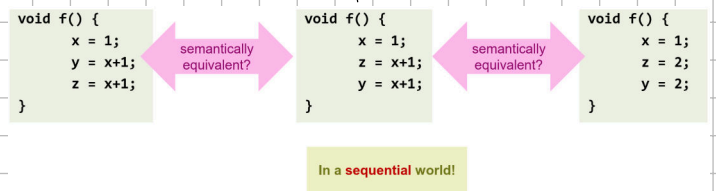
There is no interleaving of f and g that would cause the assertion to fail:



Proof by exhaustion (or full enumeration)!

→ It can still fail because of memory reordering:

→ Compiler/HW can make changes that do not affect semantics of a sequentially executed program



In a sequential world!

→ Modern compilers do not guarantee a total order of mem. accesses → many optimizations are made that are incompatible with this

Example: Fail with self-made rendezvous (C / GCC)

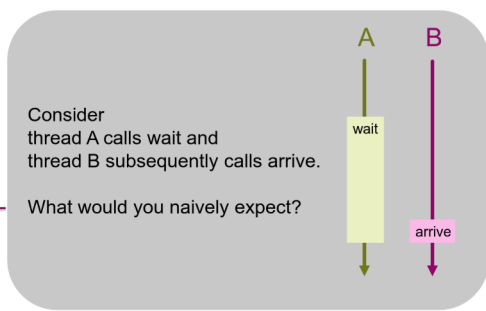
```

int x;

void wait() {
    x = 1;
    while(x==1);
}

void arrive() {
    x = 2;
}

```



Example: Fail with self-made rendezvous (C / GCC)

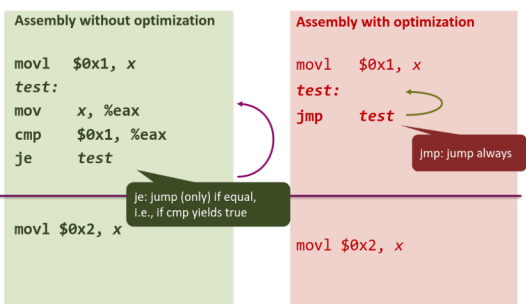
```

int x;

void wait() {
    x = 1;
    while(x==1);
}

void arrive() {
    x = 2;
}

```



→ you have probably noticed this course is deeply intertwined with some DCA concepts

→ your program that compiles to instructions is reordered at runtime by modern pipelined OoO processors

→ as each processor has a local cache, loads/stores to shared memory can be visible to diff. processors at diff. times

→ a memory model provides (often minimal) guarantees for the effects of memory ops. with optimization space for HW/compiler and guidelines for multithreaded correctness

Fixed example on next page:

→ assertion cannot fail

```

class C {
private int x = 0;
private int y = 0;
void f() {
    synchronized(this) { x = 1; }
    synchronized(this) { y = 1; }
}
void g() {
    int a, b;
    synchronized(this) { a = y; }
    synchronized(this) { b = x; }
    assert(b >= a);
}
}

```

```

class C {
private volatile int x = 0;
private volatile int y = 0;
void f() {
    x = 1;
    y = 1;
}
void g() {
    int a = y;
    int b = x;
    assert(b >= a);
}
}

```

→ volatile: accesses do not count as data races, later on you will see the cache implications

→ slower than regular fields, faster than locks

→ JMM restricts allowed program outcomes

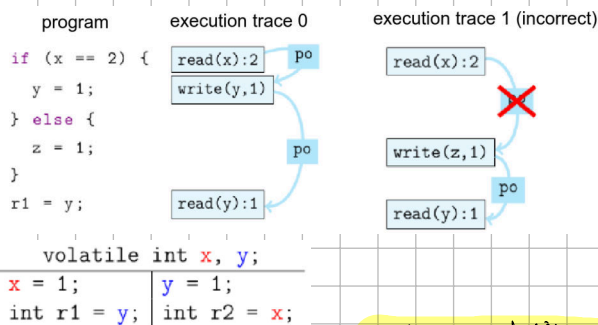
→ Actions in the JMM are things like read(x):1 ^{→ value read from field x is 1}

Program Order:

→ Total order of intra-thread actions defined on an execution trace → not total order across threads

→ Program order provides no ordering guarantee for mem. accesses → exists to link possible executions and original program

→ Intra-thread consistency: Per thread, PO order consistent with isolated thread execution



→ Synchronization Actions (SA): read/write of volatile var., lock monitor, unlock monitor, first/last action of a thread (synthetic), actions which start a thread, actions that determine if a thread has terminated

→ SA form the Synchronization Order (SO): SO is a total order, all threads see SA in same order, SA within a thread are in PO, SO is consistent: all reads in SO see last writes in SO

→ Synchronizes - With order (SW) pairs specific actions that see each other

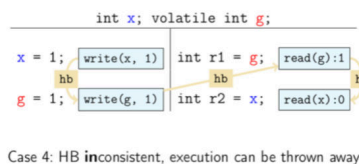
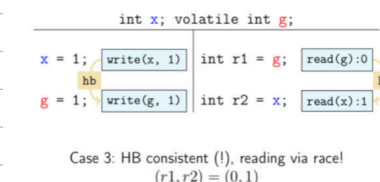
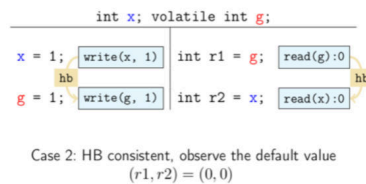
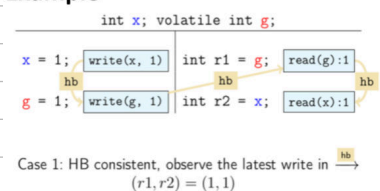
Exercise: List all outcomes (r1,r2) allowed by the JMM.

→ volatile write synchronizes with subsequent read of x (subseq. in SO)

→ transitive closure of PO and SW forms Happens-before order (HB)

→ HB consistency: when reading a variable, we either see last write in HB or other unordered write → races allowed

Example

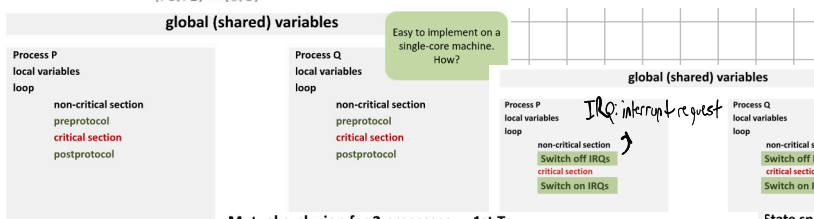


Implementing Mutex:

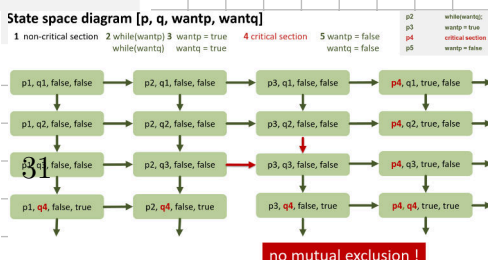
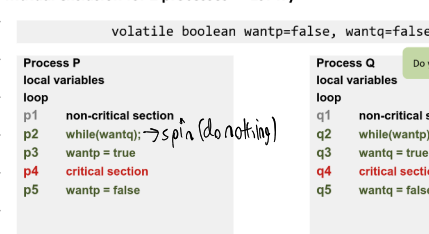
→ Assume: atomic reads/writes of primitive variables, no reordering of read/write sequences (not true in practice), threads entering CS will leave it eventually

Conditions for CS:

1. Mutex: statements from CS of 2+ processes not interleaved
2. freedom from deadlock: if some processes are trying to enter CS, one must eventually succeed
3. freedom from starvation: if any process tries to enter its CS, that process must eventually succeed



Mutual exclusion for 2 processes -- 1st Try



Observation: state space diagram too large

volatile bool

Only of interest: state transitions of the protocol.
p1/q1 is identical to p2/q2 – call state 2
p4/q4 is identical to p5/q5 – call state 5
Then forbidden: both processes in state 5

Process P
local variables
loop
p1 non-critical section
p2 while(wantq);
p3 wantp = true
p4 critical section
p5 wantp = false

Process Q
local variables
loop
q1 non-critical section
q2 while(wantp);
q3 wantq = true
q4 critical section
q5 wantq = false

Mutual exclusion for 2 processes -- 2nd Try

volatile boolean wantp=false, wantq=false

Process P
local variables
loop
p1 non-critical section
p2 wantp = true
p3 while(wantq);
p4 critical section
p5 wantp = false

Process Q
local variables
loop
q1 non-critical section
q2 wantq = true
q3 while(wantp);
q4 critical section
q5 wantq = false

Do you see the problem?

M Mutual exclusion for 2 processes -- 3rd Try

volatile int turn = 1;

Process P
local variables
loop
p1 non-critical section
p2 while(turn != 1);
p3 critical section
p4 turn = 2

Process Q
local variables
loop
q1 non-critical section
q2 while(turn != 2);
q3 critical section
q4 turn = 1

Do you see the problem?

Important, memorize these!

A combination of the tries 2 and 3: Decker's Algorithm

volatile boolean wantp=false, wantq=false, integer turn = 1

Process P
loop
non-critical section
wantp = true
while (wantq) {
if (turn == 2) {
wantp = false;
while(turn != 1);
wantp = true; }
critical section
turn = 2
wantp = false

only when q tries to get lock
and q has preference
let q proceed
and wait
and try again

Process Q
loop
non-critical section
wantq = true
while (wantp) {
if (turn == 1) {
wantq = false;
while(turn != 2);
wantq = true; }
critical section
turn = 1
wantq = false

Reduced state space diagram [p, q, wantp, wantq] – only states 2, 3, and 5

1 non-critical section 2 await wantq == false 3 wantp = true 4 critical section 5 wantp = false, wantq = false

All of interest covered:

no mutual exclusion !

State space diagram [p, q, wantp, wantq]

1 non-critical section 2 wantp = true, wantq = true 3 while(wantp), while(wantq) 4 critical section 5 wantp = false, wantq = false

deadlock !

State space diagram [p, q, turn]

We have not made any assumptions about progress outside of the CS...

starvation !

More concise than Decker: Peterson Lock (recap)

let P=1, Q=2; volatile boolean array flag[1..2] = [false, false]; volatile integer victim = 1

Process P (1)
loop
non-critical section
flag[P] = true
victim = P
while(flag[Q] && victim == Q);
critical section
flag[P] = false

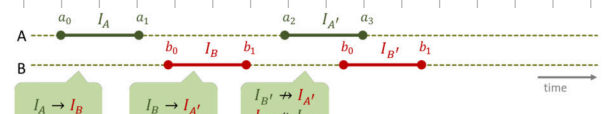
I am interested
but you go first
We both are interested
And you go first

Process Q (2)
loop
non-critical section
flag[Q] = true
victim = Q
while(flag[P] && victim == Q);
critical section
flag[Q] = false

→ you can look at the Peterson proof if you like, I'll omit it and go over the concepts used in the proof

Events and precedence: Threads produce a sequence of events → example: P produces events $p_0, p_1, \dots$ → example: $p_1 = \text{"flag[P]=true"}$
 j -th occurrence of event i in thread P : p_i^j → example: $p_2^3 = \text{"flag[P]=false"}$ in the 3rd iteration
 Precedence relation: $a \rightarrow b$ means a precedes b → total order for events

Intervals: (a_0, a_1) : interval of events a_0, a_1 with $a_0 \rightarrow a_1$, if $I_A = (a_0, a_1), I_B = (b_0, b_1), I_A \rightarrow I_B$ if $a_1 \rightarrow b_0$



we say " I_A precedes I_B " and " $I_{B'}$ and $I_{A'}$ are concurrent"

Atomic register (relevant later):

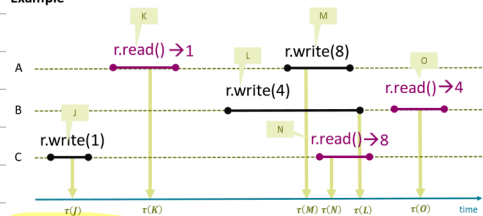
→ Register: basic mem. object, can be shared or not, not a register of a CPU

→ Register r has operations $r.read()$ and $r.write(v)$

Atomic Register: Invocation J of $r.read$ or $r.write$ takes effect at single point $\tau(J)$ in time

→ $\tau(J)$ always lies between start and end of the operation J , 2 operations J, K on same register always have different effect time $\tau(J) \neq \tau(K)$

→ Invocation J of $r.read()$ returns value v written by invocation K of $r.write(v)$ with closest preceding effect time $\tau(K)$



→ Assumptions for atomic registers allow us to treat operations on them as events at a single point in time

→ Programs can still be non-deterministic because nothing is said about order of effect times for concurrent operations

Filter Lock:

→ extension of Peterson to n processes

→ every thread knows its level in the filter level[i]

→ entering CS requires thread to traverse all levels

→ for each level: use Peterson → filter at most one thread if other threads higher level

→ for each level l: one victim victim[l] that has to let others pass in case of conflict

Peterson in Java

```
class PetersonLock {
    volatile boolean flag[] = new boolean[2];
    volatile int victim;

    public void Acquire(int id) {
        flag[id] = true;
        victim = id;
        while (flag[1-id] && victim == id);
    }

    public void Release(int id) {
        flag[id] = false;
    }
}
```

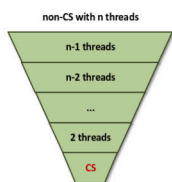
Common Bug!
Volatile reference to an array and not an array of volatile variables!
This example may work in practice. However, for a correct program we need to use to use Java's AtomicInteger and AtomicIntegerArray.

The Filter Lock

```
int[] level(#threads), int[] victim(#threads)
```

```
lock(me) {
    for (int i=1; i<n; ++i) {
        level[me] = i;
        victim[i] = me;
        while ((∃k ≠ me: level[k] >= i && victim[i] == me) {});
    }
}

unlock(me) {
    level[me] = 0;
}
```



Other threads are at same or higher level

And I have to wait

FilterLock in Java

```
import java.util.concurrent.atomic.AtomicIntegerArray;

class FilterLock {
    AtomicIntegerArray level;
    AtomicIntegerArray victim;
    volatile int n;

    FilterLock(int n) {
        this.n = n;
        level = new AtomicIntegerArray(n);
        victim = new AtomicIntegerArray(n);
    }
    ...
}
```

```
...
// ∃k ≠ me: level[k] >= i (lev)
boolean Others(int me, int lev) {
    for (int k = 0; k < n; ++k)
        if (k != me && level.get(k) >= lev) return true;
    return false;
}

public void Acquire(int me) {
    for (int lev = 1; lev < n; ++lev) {
        level.set(me, lev);
        victim.set(lev, me);
        while (me == victim.get(lev) && Others(me, lev));
    }
}

public void Release(int me) {
    level.set(me, 0);
}
```

Thread makes progress if:
a) other thread wants to enter its level
b) no threads are in front of it
→ works because of most n threads in system

Fairness: (hard to define, we will see)

→ Divide any lock implementation into 2 parts: doorway interval D: finite number of steps, waiting interval W: unbounded number of steps

→ Lock algorithm is first-come first-served if for 2 processes A, B: $D_A^i \rightarrow D_B^k \Rightarrow CS_A^i \rightarrow CS_B^k$

→ all of our locks so far of course satisfy mutex, are deadlock free, are starvation free

→ Dekker and Peterson are fair for 2 processes, the filter lock is not first-come first-serve fair

Bakery algorithm:

→ Process is required to take numbered ticket with value greater than all outstanding tickets → CS entry: wait until ticket number is lowest

Bakery algorithm (two processes, simplified)

```
volatile int np = 0, nq = 0
```

Process P
loop
non-critical section
 $np = nq + 1$
while ($nq \neq 0$ && $nq < np$);
critical section
 $np = 0$

Q also wants access

and Q has an earlier ticket

Process Q
loop
non-critical section
 $nq = np + 1$
while ($np \neq 0$ && $np < nq$);
critical section
 $nq = 0$

$np == nq$ can happen
→ global ordering of processes

Bakery Lock in Java

```
class BakeryLock {
    AtomicIntegerArray flag; // there is no
    AtomicIntegerArray label;
    final int n;

    BakeryLock(int n) {
        this.n = n;
        flag = new AtomicIntegerArray(n);
        label = new AtomicIntegerArray(n);
    }

    int MaxLabel() {
        int max = label.get(0);
        for (int i = 1; i < n; ++i)
            max = Math.max(max, label.get(i));
        return max;
    }
    ...
}
```

Nice lock! But which problem remains?

```
boolean Conflict(int me) {
    for (int i = 0; i < n; ++i)
        if (i != me && flag.get(i) != 0) {
            int diff = label.get(i) - label.get(me);
            if (diff < 0 || diff == 0 && i < me)
                return true;
        }
    return false;
}

public void Acquire(int me) {
    flag.set(me, 1);
    label.set(me, MaxLabel() + 1);
    while (Conflict(me));
}

public void Release(int me) {
    flag.set(me, 0);
}
```

Bakery algorithm (n processes)

```
integer array[0..n-1] label = [0, ..., 0]
boolean array[0..n-1] flag = [false, ..., false]
```

```
lock(me):
    flag[me] = true;
    label[me] = max(label[0], ..., label[n-1]) + 1;
    while ((∃k ≠ me: flag[k] && (k, label[k]) < (me, label[me])) {});

unlock(me):
    flag[me] = false;
```

$(k, l_k) < (j, l_j) \Leftrightarrow l_k < l_j \text{ or } (l_k = l_j \text{ and } k < j)$

Problem: atomic registers are limited → simple read/write not enough to distinguish order of writing of N threads → we only see last write

→ to differentiate n threads, you need n variables (example: victim above)

→ Mutex is not implemented like this, especially because the space lower bound is linear in the amount of maximum threads

→ How can we do better? leverage special ins. in our architectures!

Test and Set:

Compare and Set:

boolean TAS(memref s)

```
if (mem[s] == 0) {
    mem[s] = 1;
    return true;
} else
    return false;
```

int CAS (memref a, int old, int new)

```
oldval = mem[a];
if (old == oldval)
    mem[a] = new;
return oldval;
```

→ Atomic operations = Read-Modify-Write, you will see them everywhere
implementation of a spinlock using simple atomic operations

Test and Set (TAS)

```
Init (lock)
lock = 0;

Acquire (lock)
while !TAS(lock); // wait

Release (lock)
lock = 0;
```

Compare and Swap (CAS)

```
Init (lock)
lock = 0;

Acquire (lock)
while (CAS(lock, 0, 1) != 0);

Release (lock)
CAS(lock, 1, 0);
```

ignore result

→ Java supports atomic operations using the library `java.util.concurrent.atomic` (standard)

→ Atomic Boolean in that library has operations:

`boolean set()`, `boolean get()`, `boolean compareAndSet(boolean expect, boolean update)`, `boolean getAndSet(boolean newValue)`
→ automatically set to value update if the current value is expect, return true on success
→ sets new value and returns previous value

Spinlock: Try to get lock, keep trying until lock acquired (return value is false), unlock, release lock (set to false)

→ TASLock (a spinlock) in Java:

```
public class TASLock implements Lock {
    AtomicBoolean state = new AtomicBoolean(false);

    public void lock() {
        while(!state.getAndSet(true)) {}
    }

    public void unlock() {
        state.set(false);
    }
    ...
}
```

→ this linearly slows down with more processes → sequential bottleneck

→ causes: contention → threads fight for shared bus during `getAndSet` call
cache coherence protocol invalidates cached copies of lock on other processors

Test-And-Test-And-Set (TTAS) lock:

```
public void lock()
{
    do
        while(state.get()) {}
        while (!state.compareAndSet(false, true));
    }

    public void unlock()
    {
        state.set(false);
    }
}
```

→ scales measurably better

→ still, contention is a problem

→ solution: send threads to sleep with random duration, increase expected duration each time resource not free

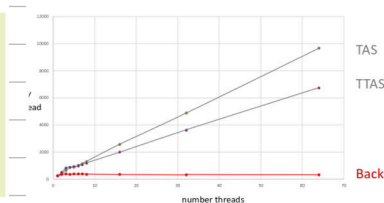
Lock with Backoff

```
public void lock() {
    Backoff backoff = null;
    while (true) {
        while (state.get()) {} // spin reading only (TTAS)
        if (!state.getAndSet(true)) // try to acquire, returns previous val
            return;
        else { // backoff on failure
            try {
                if (backoff == null) // allocation only on demand
                    backoff = new Backoff(MIN_DELAY, MAX_DELAY);
                backoff.backoff();
            } catch (InterruptedException ex) {}
        }
    }
}
```

exponential backoff

```
class Backoff {
    ...
    public void backoff() throws InterruptedException {
        int delay = random.nextInt(limit);
        if (limit < maxDelay) { // double limit if less than max
            limit = 2 * limit;
        }
        Thread.sleep(delay);
    }
}
```

Measurement



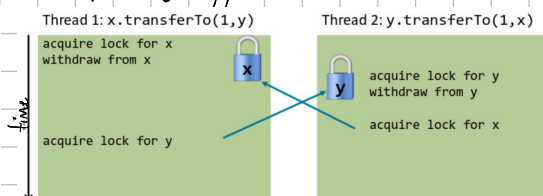
Deadlock:

Motivation: (Re)consider the bank account class: → Assume x, y are instances of `BankAccount`

```
class BankAccount {
    ...
    synchronized void withdraw(int amount) {...}
    synchronized void deposit(int amount) {...}

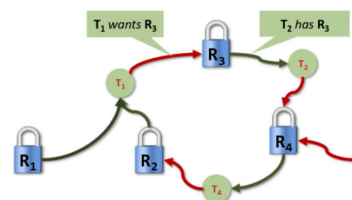
    synchronized void transferTo(int amount, BankAccount a) {
        this.withdraw(amount);
        a.deposit(amount);
    }
}
```

Thread acquires second lock in `a.deposit`
Can this become a problem?



Deadlock: 2+ processes mutually blocked because each process waits for another to proceed → recall the Dining Philosophers Problem

More formally: a deadlock for threads $T_1, \dots, T_n$ occurs when the directed graph describing relation of $T_1, \dots, T_n$ and resources $R_1, \dots, R_n$ contains cycle



→ in plain English: a circular dependency of locks

→ Intuitively enough, in systems, deadlock detection is done by finding cycles in the dependency graph

→ Deadlocks can generally not be healed → releasing locks generally leads to inconsistent state

Deadlock avoidance (below):

→ 2-phase locking with retry (release when failed) → usually in databases where transactions can be aborted without consequence

→ resource ordering: usual in parallel programming where state is modified

→ Coding part of Exercise 9 is great to solidify this

→ Fixing the prior example:

Option 1: non-overlapping (smaller) CS

```
class BankAccount {
    ...
    synchronized void withdraw(int amount) {...}
    synchronized void deposit(int amount) {...}
    ...
    void transferTo(int amount, BankAccount a) {
        this.withdraw(amount);
        a.deposit(amount);
    }
}
```

Money disappears for a (very short?) moment!
Can we allow such transient inconsistencies?
Very often unacceptable!

Option 2: 1 lock for all

```
class BankAccount {
    static Object globalLock = new Object();
    // withdraw and deposit protected with globalLock!
    void withdraw(int amount) {...}
    void deposit(int amount) {...}
    ...
    void transferTo(int amount, BankAccount to) {
        synchronized (globalLock) {
            withdraw(amount);
            to.deposit(amount);
        }
    }
}
```

deadlock avoided but no concurrent transfer possible, even not when the pairs of accounts are disjoint.
Often very inefficient!

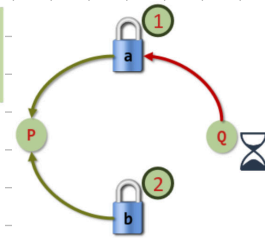
Option 3: global ordering of resources

```
class BankAccount {
    ...
    void transferTo(int amount, BankAccount to) {
        if (to.accountNr < this.accountNr)
            synchronized(this) {
                withdraw(amount);
                to.deposit(amount);
            }
        else
            synchronized(to) {
                withdraw(amount);
                to.deposit(amount);
            }
    }
}
```

Unique global ordering required.
Whole program has to obey this order to avoid cycles.
Code taking only one lock can ignore it.

Programming trick: Generate unique global order:

```
class BankAccount {
    private static final AtomicLong counter = new AtomicLong();
    private final long index = counter.incrementAndGet();
    ...
    void transferTo(int amount, BankAccount to) {
        if (to.index < this.index)
            ...
    }
}
```



→ For this kind of concurrent system, deadlocks are hard to work around → actually dominant problem of complex concurrent systems

→ Easier cures: different types of objects → document fixed order among types, acyclic object structure → use data structure to determine order (example: tree nodes)

→ Starvation: repeated unsuccessful attempt of unblocked process to continue executing

Semaphores:

→ we need more than locks: they provide means to enforce atomicity via mutex → don't allow threads to communicate about state change
→ provide no order, hard to use (what if 2 threads A, B lock an object? which one executes first?)

Semantics: Semaphore → integer-valued abstract data type S with initial value $s \geq 0$ and the following operations:

```
acquire(S) {
    atomic {
        wait until s > 0
        dec(S)
    }
}

release(S) {
    atomic {
        inc(S)
    }
}
```

→ A lock is just a binary semaphore:

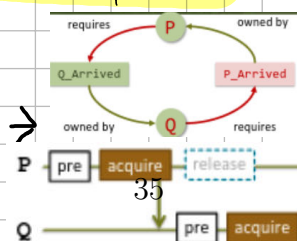
→ $\text{sem_mutex} = \text{Semaphore}(1)$; $\text{lock_mutex} := \text{sem_mutex.acquire}()$; $\text{unlock} := \text{sem_mutex.release}()$
→ 1 thread allowed into CS (only) → 1 thread let in

→ Semaphore number: 1 → unlocked, 0 → locked, $x > 0$ → x threads let into "CS"

Rendezvous with Semaphores: 2 processes P, Q executing code → Rendezvous: locations in code where P, Q wait for the other to arrive, synchronize them

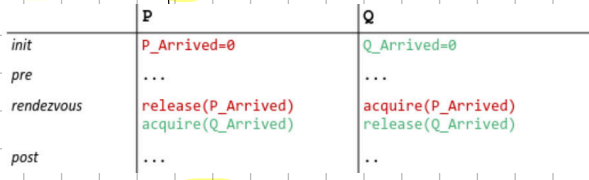
→ Create semaphores P_Arrived and Q_Arrived → First attempt below:

	P	Q
init	P_Arrived=0	Q_Arrived=0
pre	...	...
rendezvous	acquire(Q_Arrived) release(P_Arrived)	acquire(P_Arrived) release(Q_Arrived)
post	...	...



→ Deadlock, informally I would say "acquisition too greedy"

2nd attempt:



Aside: Semaphores without Spinning → consider process list Q_s associated with semaphore S :

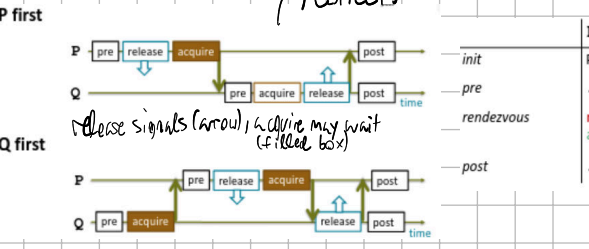
```

acquire(S)
{
  if S > 0 then
    dec(S)
  else
    put(Qs, self)
    block(self)
  end
}

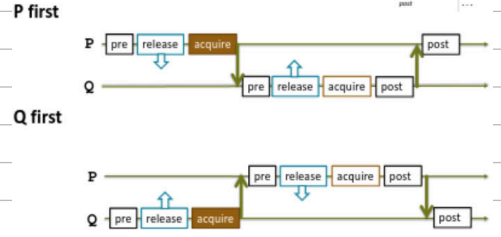
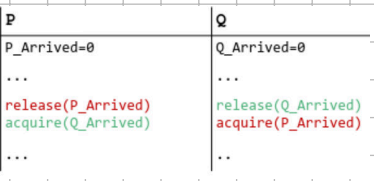
release(S)
{
  if Qs == 0 then
    inc(S)
  else
    get(Qs, p)
    unblock(p)
  end
}
    
```

Scheduling Scenarios:

→ correct!



3rd attempt → even better:

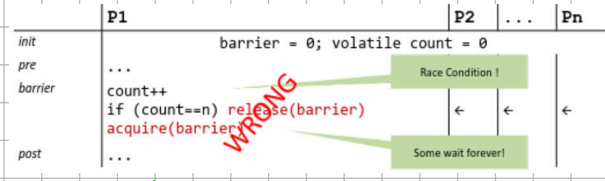


Barriers: Synchronize a number of processes:



Implementing it via semaphores: synchronize n processes, Semaphore barrier, Integer count:

1st attempt:

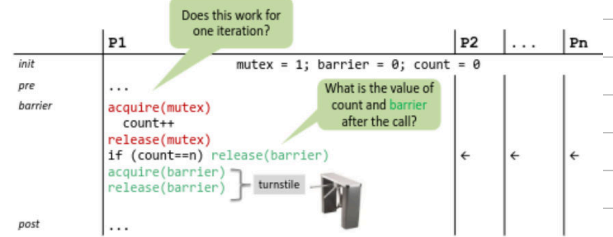
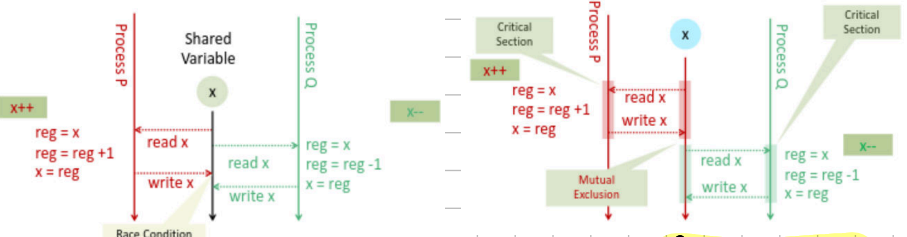


Why not? Invariants: each process reaches acquire eventually ✓, barrier only opens if all processes have reached it ✓, count gives number of processes that have passed barrier ✗, all waiting processes can continue when all processes have reached the barrier ✗

Recall: race condition on "count++":

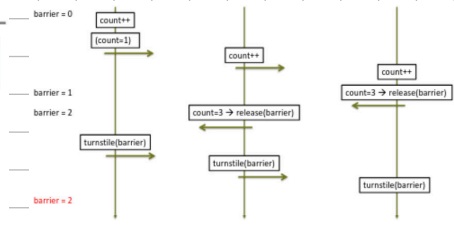
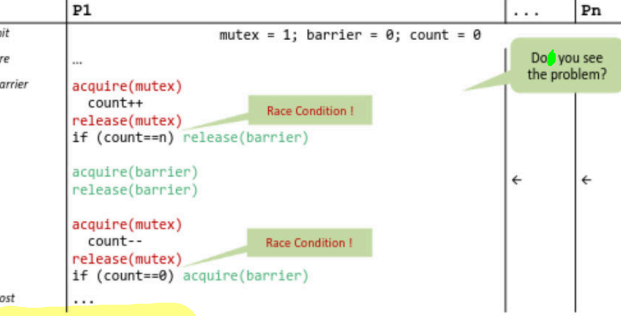
With mutual exclusion:

2nd attempt:



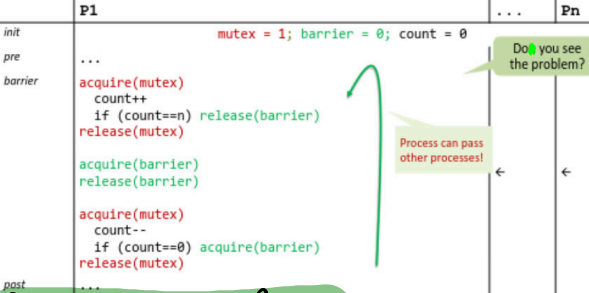
⇒ Reusable Barrier, 1st attempt:

Problem illustrated:



Invariants: Turnstile only opens when all processes have reached it ✓, count=0 when all processes go through barrier ✓, barrier=0 when all proc. go through barrier ✗

2nd attempt:



Invariants:

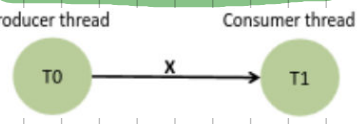
barrier=0 when all processes pass barrier ✓, even when single process passes barrier, barrier=0 ✗

Solution: 2-phase barrier (slow in practice)

```

init
barrier
mutex=1; barrier1=0; barrier2=1; count=0
acquire(mutex)
count++
if (count==n) {
  acquire(barrier2); release(barrier1)
  release(mutex)
}
acquire(barrier1); release(barrier1);
// barrier1 = 1 for all processes, barrier2 = 0 for all processes
acquire(mutex)
count--
if (count==0) {
  acquire(barrier1); release(barrier2)
  release(mutex)
}
acquire(barrier2); release(barrier2)
// barrier1 = 0 for all processes, barrier2 = 1 for all processes
    
```

Producer-Consumer Routine:

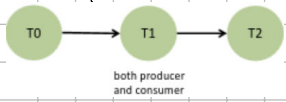


→ T_0 computes x , passes it to T_1

→ T_1 uses x , does x have to be synchronized?

→ not at any point in time, one thread accesses x (only), passing x must be synchronized though

Example: Pipelines

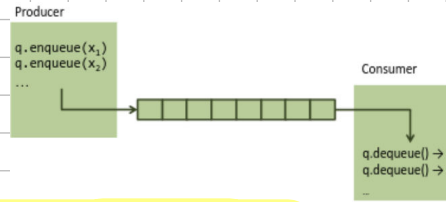


Pipeline Node:

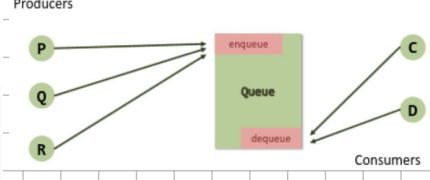
```
while (true) {
    input = q.in.dequeue();
    output = do_something(input);
    q.out.enqueue(output);
}
```



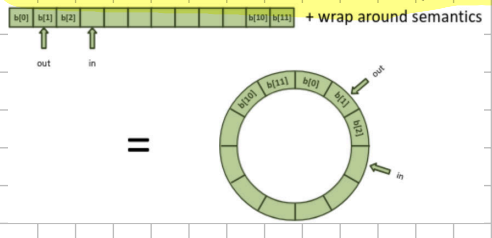
Producer/Consumer Queue:



Multiple Producers/Consumers:



Bounded FIFO as a Circular Buffer:



Prod./Con. Queue Implementation

```
class Queue {
    private int in; // next new element
    private int out; // next element
    private int size; // queue capacity
    private long[] buffer;

    Queue(int size) {
        this.size = size;
        in = out = 0;
        buffer = new long[size];
    }

    private int next(int i) {
        return (i + 1) % size;
    }
}
```

```
public synchronized void enqueue(long item) {
    buffer[in] = item;
    in = next(in);
}

public synchronized long dequeue() {
    item = buffer[out];
    out = next(out);
    return item;
}
```

What if we try to
1. dequeue from an empty queue?
2. enqueue to a full queue?

Helper Function:

```
public void doEnqueue(long item) {
    buffer[in] = item;
    in = next(in);
}

public boolean isFull() {
    return (in+1) % size == out;
}
```

```
public long doDequeue() {
    long item = buffer[out];
    out = next(out);
    return item;
}

public boolean isEmpty() {
    return in == out;
}
```

how to implement helpers?

```
public synchronized void enqueue(long item) {
    while (isFull())
        ; // wait
    doEnqueue(item);
}
```

```
public synchronized long dequeue() {
    while (isEmpty())
        ; // wait
    return doDequeue();
}
```

Do you see the problem?

→ blocks forever, infinite loops with a lock held

Producer/Consumer with semaphores:

```
import java.util.concurrent.Semaphore;

class Queue {
    int in, out, size;
    long buf[];
    Semaphore nonEmpty, nonFull, manipulation;

    Queue(int s) {
        size = s;
        buf = new long[size];
        in = out = 0;
        nonEmpty = new Semaphore(0); // use the counting feature of semaphores!
        nonFull = new Semaphore(size); // use the counting feature of semaphores!
        manipulation = new Semaphore(1); // binary semaphore
    }

    public void enqueue(long item) throws InterruptedException {
        while (true) {
            synchronized(this) {
                if (!isFull()) {
                    doEnqueue(item);
                    return;
                }
            }
            Thread.sleep(timeout); // sleep without lock!
        }
    }

    public long dequeue() {
        long x = 0;
        try {
            nonEmpty.acquire();
            manipulation.acquire();
            x = buf[out];
            out = next(out);
        } catch (InterruptedException ex) {}
        finally {
            manipulation.release();
            nonFull.release();
        }
        return x;
    }
}
```

Using sleep():

```
public void enqueue(long item) throws InterruptedException {
    while (true) {
        synchronized(this) {
            if (!isFull()) {
                doEnqueue(item);
                return;
            }
        }
        Thread.sleep(timeout); // sleep without lock!
    }
}
```

What is the proper value for the timeout?
Ideally we would like to be notified when the change happens!
When is that?

Producer / Consumer queues with semaphores

```
void enqueue(long x) {
    try {
        nonFull.acquire();
        manipulation.acquire();
        buf[in] = x;
        in = next(in);
    } catch (InterruptedException ex) {}
    finally {
        manipulation.release();
        nonEmpty.release();
    }
}
```

```
long dequeue() {
    long x = 0;
    try {
        nonEmpty.acquire();
        manipulation.acquire();
        x = buf[out];
        out = next(out);
    } catch (InterruptedException ex) {}
    finally {
        manipulation.release();
        nonFull.release();
    }
    return x;
}
```

→ Semaphores are problematic → unstructured, can easily cause deadlock

→ We need a lock we can temporarily escape while waiting on a condition

→ Before we move on, I urge you to take a look at Exercise 10 - Dining Philosophers

Monitors:

→ abstract data structure equipped with set of operations running in mutual exclusion

Monitors vs. Semaphores/Unbound Locks



For Prod./Con. Queue example:

```
public void synchronized enqueue(long item) {
    "while (isFull()) wait"
    doEnqueue(item);
}

public long synchronized dequeue() {
    "while (isEmpty()) wait"
    return doDequeue();
}
```

The mutual exclusion part is nicely available already. But: while the buffer is full we need to give up the lock, how?

→ Monitors have the following semantics: If a condition does not hold, we can: release the monitor lock, wait for the condition to become true, signal to avoid busy loops (spinning)

→ In Java, the simplest implementation of monitors is the intrinsic lock (synchronized) + wait()/notify()/notifyAll(), which we know

Prod./Con. Queue with Monitors:

```
class Queue {
    int in, out, size;
    long buf[];

    Queue(int s) {
        size = s;
        buf = new long[size];
        in = out = 0;
    }
    ...
}
```

```
synchronized void enqueue(long x) {
    while (isFull())
        try {
            wait();
        } catch (InterruptedException e) {}
    doEnqueue(x);
    notifyAll();
}
```

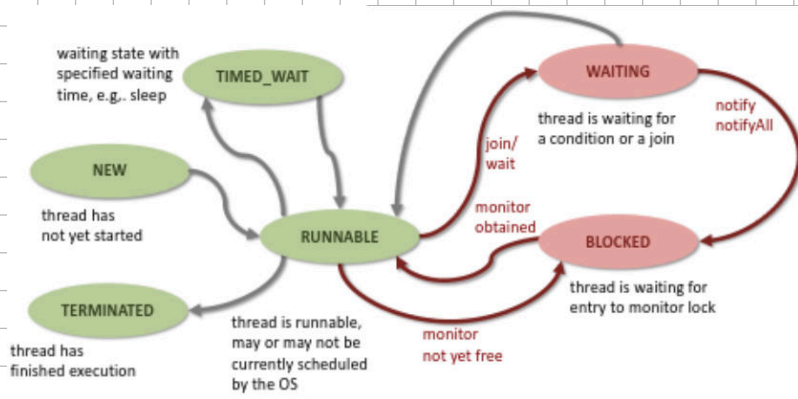
```
synchronized long dequeue() {
    long x;
    while (isEmpty())
        try {
            wait();
        } catch (InterruptedException e) {}
    x = doDequeue();
    notifyAll();
    return x;
}
```

Wouldn't an if be sufficient? → defeats the point of waiting

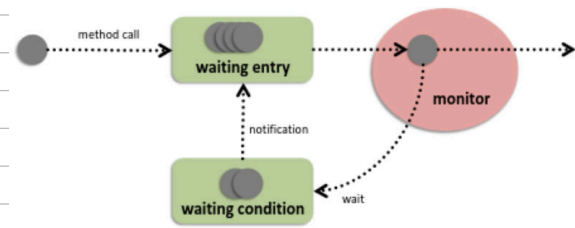
(Why) can't we use notify()? → multiple threads may be waiting

Java monitor implementation details:

Thread states:



Monitor Queues:



Exact semantics → what happens upon notification?

Signal and wait: signalling process exits monitor (goes to wait entry queue)
signalling process passes monitor lock to signalled process

Signal and continue: signalling process continues running
signalling process moves signalled process to wait entry queue

→ other semantics: signal and exit, signal and urgent wait

Signal and continue:

Scenario:

1. process P already entered semaphore and decreased number to 0.
2. process Q sees number = 0, goes to waitlist
3. P executes exit. In this moment, Q wants to enter monitor via method enter
4. P signals Q, moves it to wait entry list (signal & cont.). P exits function/lock
5. Q gets entry to monitor before Q sees number = 1
6. Q continues exec. with number = 0 → inconsistency

Try implementing a semaphore from monitors:

```
class Semaphore {
    int number = 1; // number of threads allowed in critical section

    synchronized void acquire() {
        if (number <= 0)
            try { wait(); } catch (InterruptedException e) { };
        number--;
    }

    synchronized void release() {
        number++;
        if (number > 0)
            notify();
    }
}
```

Looks good, doesn't it?
But there is a problem.
Do you know which?

as you might have picked up on, we can solve this with a while loop

```
synchronized void acquire() {
    while (number <= 0)
        try { wait(); }
    catch (InterruptedException e) { };
    number--;
}
```

```
synchronized void release() {
    number++;
    if (number > 0)
        notify();
}
```

```
R synchronized void acquire() {
    if (number <= 0)
        try { wait(); } Q
    catch (InterruptedException e) { };
    number--;
}

P synchronized void release() {
    number++;
    if (number > 0)
        notify();
}
```

→ If different threads evaluated different conditions, the notification would have to be a notifyAll()

→ Intrinsic lock should be our first choice in practice, still, it has limitations

→ 1 implicit lock per object, we forced to be used in blocks, less flexible → Lock interface (e.g. ReentrantLock())

→ Condition interface: Java locks provide conditions that can be instantiated

Condition notFull = lock.newCondition(); → Methods: **await()** → current thread waits until condition is signalled;
signal() → wakes up 1 thread waiting on this condition;
signalAll() → wakes up all threads waiting on this condition

→ Condition interface: Conditions are always associated with lock

→ **await()** → called with held lock, atomically releases lock and waits until thread is signalled, on return, it is guaranteed to hold the lock;
thread always needs to check condition

→ **signalAll()** → wakes up 1(all) waiting thread(s) → called with lock held

Prod./Con. Queue with explicit lock:

```
class Queue {
    int in=0, out=0, size;
    long buf[];
    final Lock lock = new ReentrantLock();
    final Condition notFull = lock.newCondition();
    final Condition notEmpty = lock.newCondition();

    Queue(int s) {
        size = s;
        buf = new long[size];
    }
}
```

```
void enqueue(long x) {
    lock.lock();
    while (isFull())
        try {
            notFull.await();
        } catch (InterruptedException e) {}
    doEnqueue(x);
    notEmpty.signal();
    lock.unlock();
}
```

```
long dequeue() {
    long x;
    lock.lock();
    while (isEmpty())
        try {
            notEmpty.await();
        } catch (InterruptedException e) {}
    x = doDequeue();
    notFull.signal();
    lock.unlock();
    return x;
}
```

→ Sleeping Barber variant ³⁸ (Dijkstra): Disadvantage: notFull, notEmpty signals also sent if no thread is waiting
→ additional counters to check if processes are waiting: